

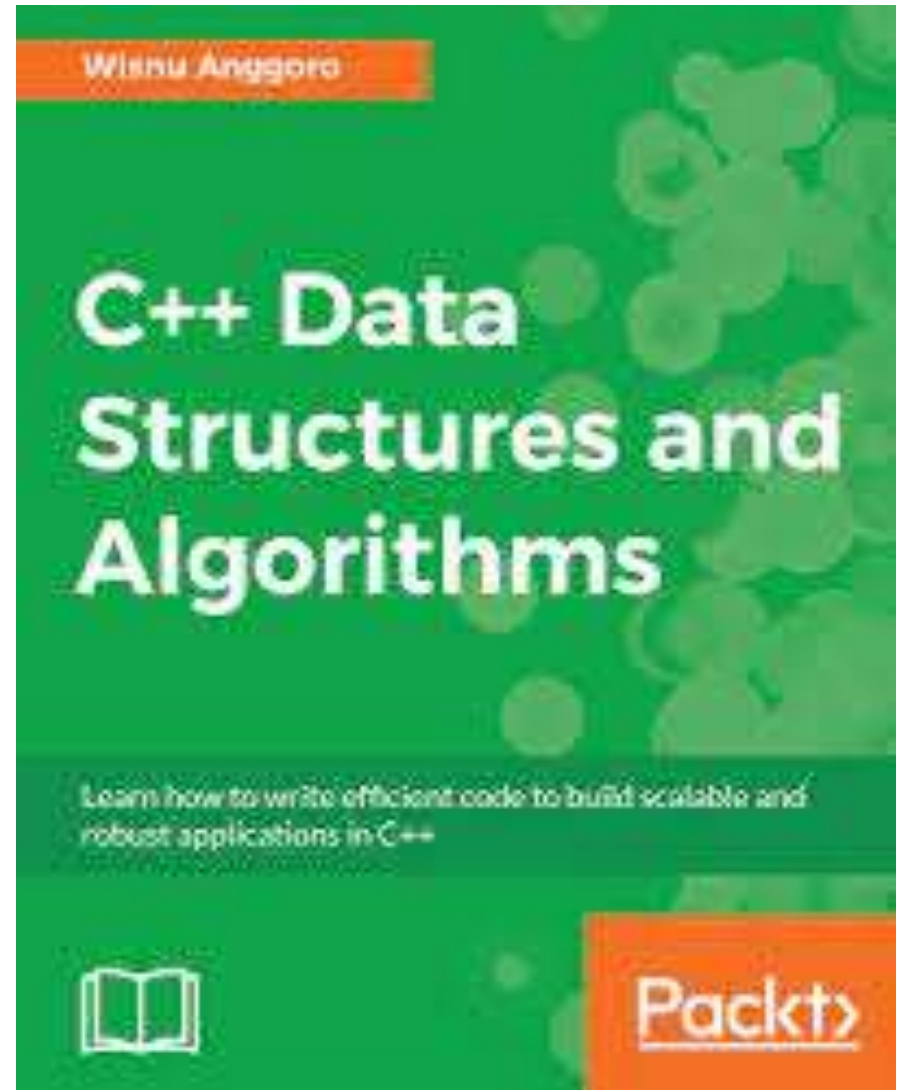
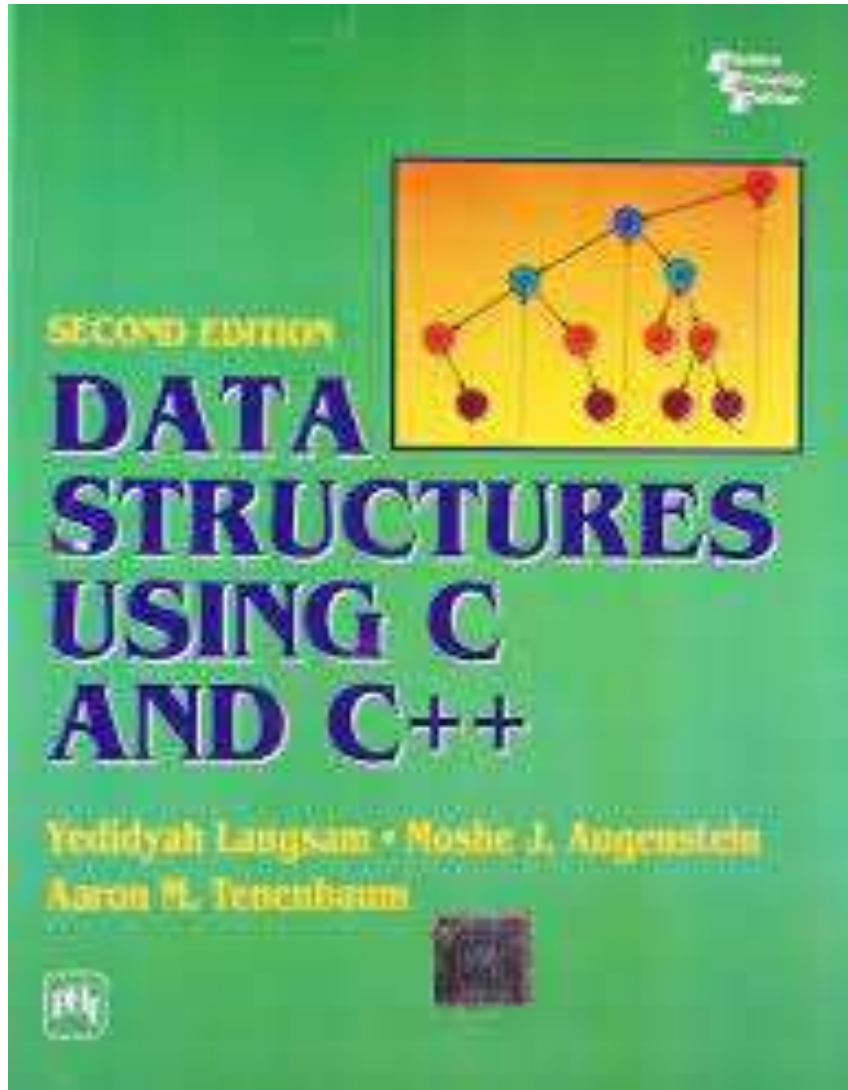


**Faculty of Computers and Information -
Department of AI-SAMS**

&

The French University in Egypt (UFE)

Text books



Review C++ Concepts I

Review C++ Concepts I

1. *Control Statements*

```
if ( statement ) {  
    // ...  
} else if ( statement ) {  
    // ...  
} else {  
    // ...  
}
```

```
while ( statement ) {  
    // ...  
}
```

```
for ( int i = 0; i < N; ++i ) {  
    // ...  
}
```

```
do {  
    // ...  
} while ( statement );
```

Review C++ Concepts I

1. Control Statements (*Cont.*)- `if` Statement

```
if (x == 100)
    cout << "x is 100";
```

```
if (x == 100)
{
    cout << "x is ";
    cout << x;
}
```

```
if (x == 100)
    cout << "x is 100";
else
    cout << "x is not 100";
```

```
if (x > 0)
    cout << "x is positive";
else if (x < 0)
    cout << "x is negative";
else
    cout << "x is 0";
```

Review C++ Concepts I

1. Control Statements (*Cont.*)- **while** Statement

```
// custom countdown using while

#include <iostream>
using namespace std;

int main ()
{
    int n;
    cout << "Enter the starting number > ";
    cin >> n;

    while (n>0) {
        cout << n << ", ";
        --n;
    }

    cout << "FIRE!\n";
    return 0;
}
```

```
Enter the starting number > 8
8, 7, 6, 5, 4, 3, 2, 1, FIRE!
```

Review C++ Concepts I

1. Control Statements (*Cont.*)- **do . . . while** *Statement*

```
// number echoer

#include <iostream>
using namespace std;

int main ()
{
    unsigned long n;
    do {
        cout << "Enter number (0 to end): ";
        cin >> n;
        cout << "You entered: " << n << "\n";
    } while (n != 0);
    return 0;
}
```

```
Enter number (0 to end): 12345
You entered: 12345
Enter number (0 to end): 160277
You entered: 160277
Enter number (0 to end): 0
You entered: 0
```

Review C++ Concepts I

1. Control Statements (*Cont.*)- *for* Statement

```
// countdown using a for loop
#include <iostream>
using namespace std;
int main ()
{
    for (int n=10; n>0; n--) {
        cout << n << ", ";
    }
    cout << "FIRE!\n";
    return 0;
}
```

10, 9, 8, 7, 6, 5, 4, 3, 2, 1, FIRE!

Review C++ Concepts I

2. Jump Statements - *break* Statement

- Using **break** we can *leave a loop* even if the condition for its end is not fulfilled.
- It can be used to end *an infinite loop*, or to *force it to end* before its natural end.

```
// break loop example

#include <iostream>
using namespace std;

int main ()
{
    int n;
    for (n=10; n>0; n--)
    {
        cout << n << ", ";
        if (n==3)
        {
            cout << "countdown aborted!";
            break;
        }
    }
    return 0;
}
```

```
10, 9, 8, 7, 6, 5, 4, 3, countdown aborted!
```

Review C++ Concepts I

2. Jump Statements (*Cont.*)- **continue** Statement

- The **continue** statement causes the program to *skip the rest of the loop* in the current iteration as if the end of the statement block had been reached, causing it to jump to the start of the following iteration.

```
// continue loop example
#include <iostream>
using namespace std;

int main ()
{
    for (int n=10; n>0; n--) {
        if (n==5) continue;
        cout << n << ", ";
    }
    cout << "FIRE!\n";
    return 0;
}
```

10, 9, 8, 7, 6, 4, 3, 2, 1, FIRE!

Review C++ Concepts I

2. Jump Statements (*Cont.*)- `goto` Statement

- `goto` allows to make an absolute jump to another point in the program.
- You should use this feature with caution since its execution causes an unconditional jump ignoring any type of nesting limitations.
- The destination point is identified by a label, which is then used as an argument for the `goto` statement.
- A label is made of a valid identifier followed by a colon (:)

Review C++ Concepts I

2. Jump Statements (*Cont.*)- goto Statement- *Example*

```
// goto loop example
```

```
#include <iostream>
using namespace std;
```

```
int main ()
```

```
{
```

```
    int n=10;
```

```
    loop:
```

```
    cout << n << ", ";
```

```
    n--;
```

```
    if (n>0) goto loop;
```

```
    cout << "FIRE!\n";
```

```
    return 0;
```

```
}
```

```
10, 9, 8, 7, 6, 5, 4, 3, 2, 1, FIRE!
```

Review C++ Concepts I

3. Selective Statements (*Cont.*)- **switch** *Statement*

- The syntax of the **switch** statement is a bit peculiar. Its objective is to check several possible constant values for an expression.
- Something similar to what we did at the beginning of this section with the concatenation of several if and else if instructions. Its form is the following:

```
switch (expression)
{
    case constant1:
        group of statements 1;
        break;
    case constant2:
        group of statements 2;
        break;
    .
    .
    .
    default:
        default group of statements
}
```

Review C++ Concepts I

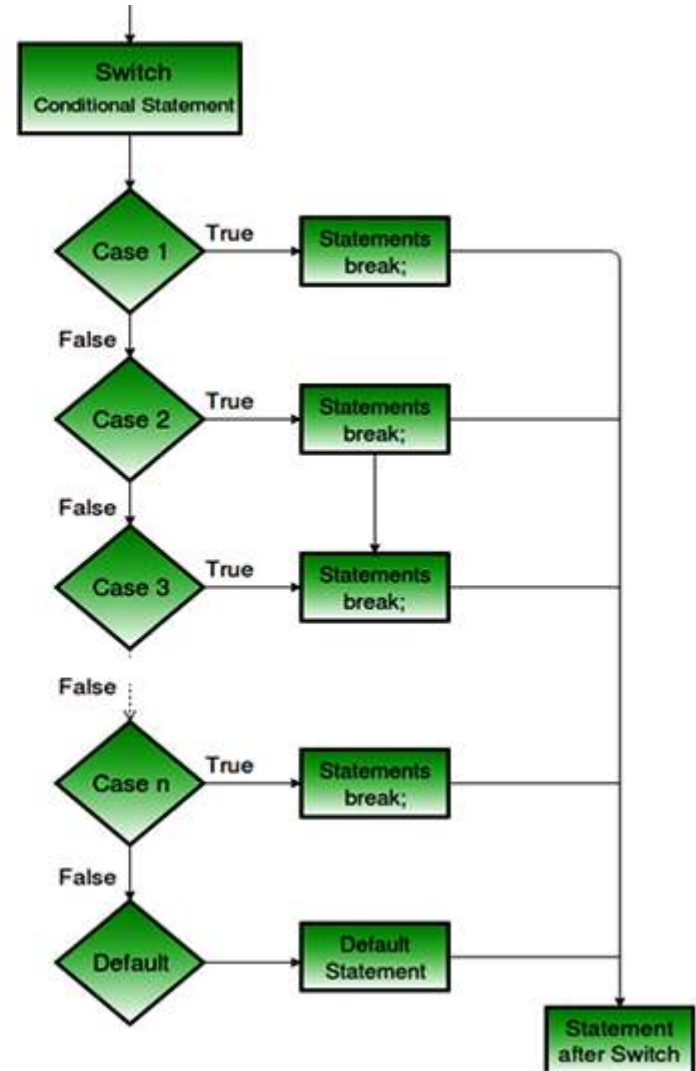
3. Selective Statements (*Cont.*)- switch Statement- *Example1*

```
// Following is a simple C program
// to demonstrate syntax of switch.
#include <stdio.h>
int main()
{
    int x = 2;
    switch (x)
    {
        case 1: printf("Choice is 1");
                break;
        case 2: printf("Choice is 2");
                break;
        case 3: printf("Choice is 3");
                break;
        default: printf("Choice other than 1, 2 and 3");
                break;
    }
    return 0;
}
```

Review C++ Concepts I

3. Selective Statements (*Cont.*)- switch & if Statements

switch example	if-else equivalent
<pre>switch (x) { case 1: cout << "x is 1"; break; case 2: cout << "x is 2"; break; default: cout << "value of x unknown"; }</pre>	<pre>if (x == 1) { cout << "x is 1"; } else if (x == 2) { cout << "x is 2"; } else { cout << "value of x unknown"; }</pre>



Review C++ Concepts I

4. *Operators*

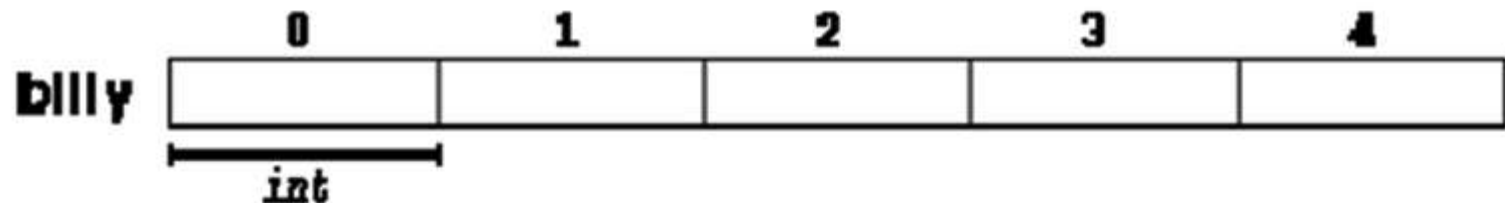
Operators have similar functionality for built-in data types:

- | | | | | | | |
|-----------------|-----|----------------|----|----|----|----|
| – Assignment | = | | | | | |
| – Arithmetic | + | - | * | / | % | |
| | += | -= | *= | /= | %= | |
| – Autoincrement | ++ | | | | | |
| – Autodecrement | -- | | | | | |
| – Logical | && | | ! | | | |
| – Relational | == | != | < | <= | >= | > |
| – Comments | /* | | | | | */ |
| | // | to end of line | | | | |
| – Bitwise | & | | ^ | ~ | | |
| | &= | = | ^= | | | |
| – Bit shifting | << | >> | | | | |
| | <<= | >>= | | | | |

Review C++ Concepts I

5. Arrays

- An **array** is a series of elements of the same type placed in contiguous memory locations that can be individually referenced by adding an index to a unique identifier.
- *For example*, an array to contain 5 integer values of type **int** called **billly** could be represented like this:



- A typical declaration for an array in C++ is:

```
type name [elements];
```

```
int billly [5];
```

Review C++ Concepts I

5. Arrays (Cont.)

- Initializing arrays

```
int billy [5] = { 16, 2, 77, 40, 12071 };
```

	0	1	2	3	4
billy	16	2	77	40	12071

- Accessing the values of an array

	billy[0]	billy[1]	billy[2]	billy[3]	billy[4]
billy					

```
billy[2] = 75;
```

Review C++ Concepts I

5. Arrays (Cont.)- Example

```
// arrays example
#include <iostream>
using namespace std;

int billy [] = {16, 2, 77, 40, 12071};
int n, result=0;

int main ()
{
    for ( n=0 ; n<5 ; n++ )
    {
        result += billy[n];
    }
    cout << result;
    return 0;
}
```

12206

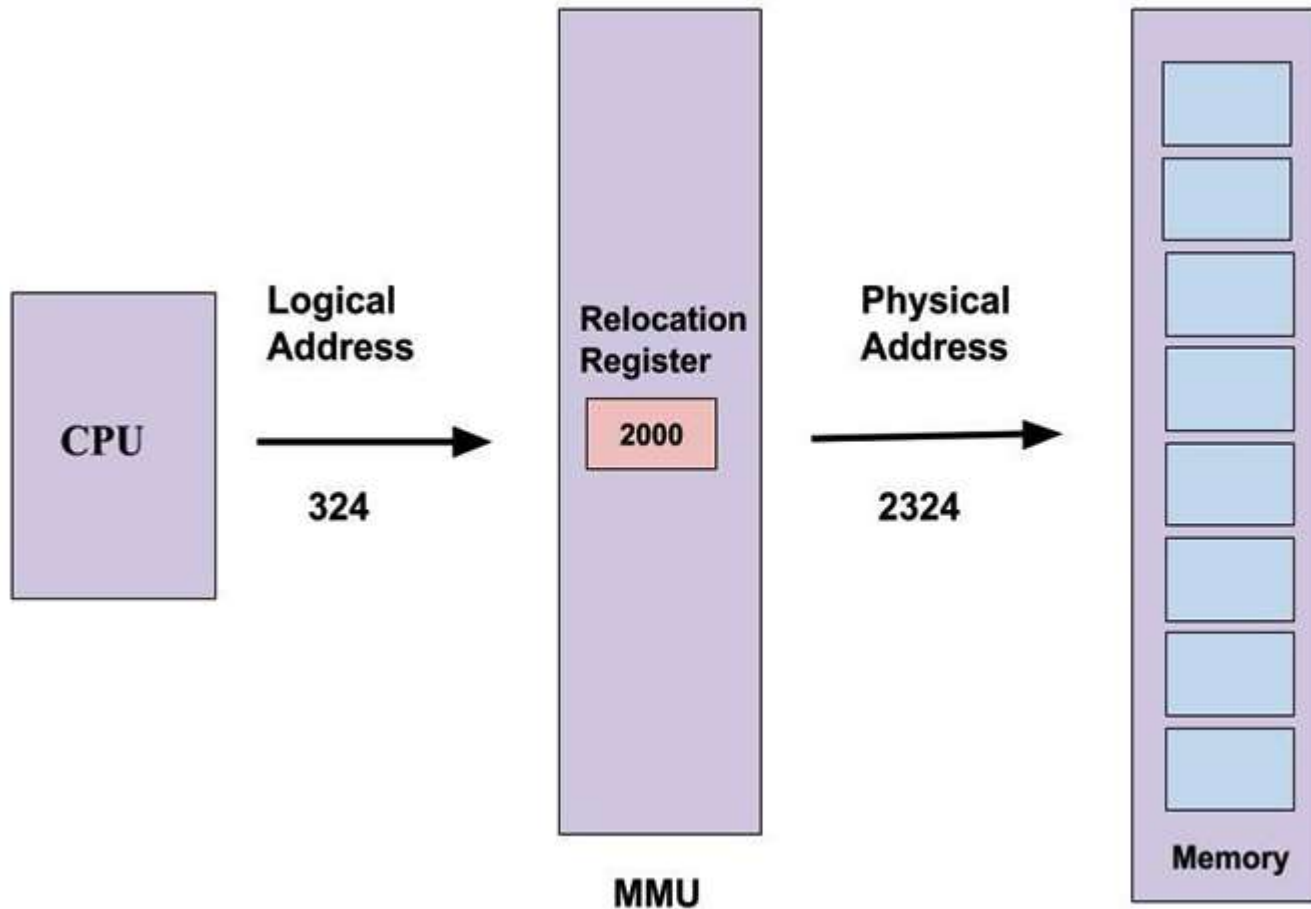
Review C++ Concepts I

5. Arrays (Cont.)- Multidimensional Array

multidimensional array
<pre>#define WIDTH 5 #define HEIGHT 3 int jimmy [HEIGHT][WIDTH]; int n,m; int main () { for (n=0;n<HEIGHT;n++) for (m=0;m<WIDTH;m++) { jimmy[n][m]=(n+1) * (m+1); } return 0; }</pre>

Review C++ Concepts I

Logical & Physical Addresses



Review C++ Concepts I

6. Pointers- Reference Operator (&)

- It may be interesting knowing the **address (physical address)** where our variable is being stored during runtime in order to operate with relative positions to it.
- *The address that locates a variable within memory is called a reference to that variable.*
- This reference to a variable can be obtained by preceding the identifier of a variable with an **ampersand sign (&)**, known as **reference operator**, and which can be literally translated as "**address of**".
 - *For example: **ted = &andy;***
- This would assign to **ted** the address of variable **andy**, since when preceding the name of the variable **andy** with the **reference operator (&)** we are no longer talking about the content of the variable itself, but about **its reference** (i.e., **its address in memory**)

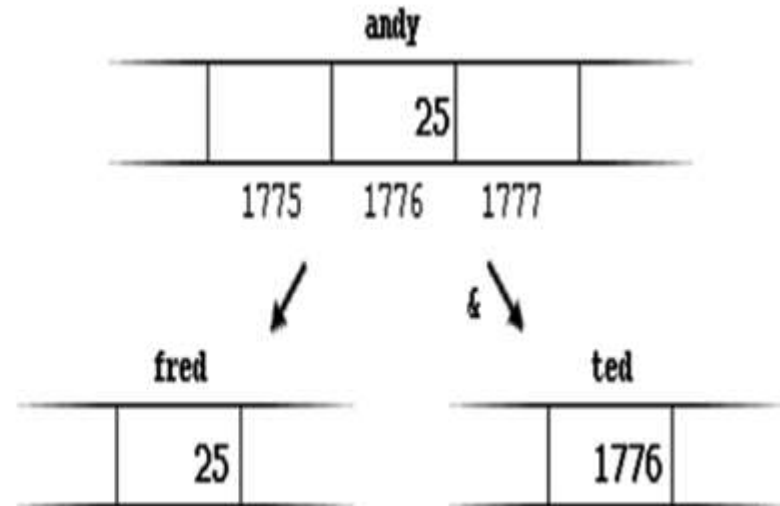
Review C++ Concepts I

6. Pointers(*Cont.*)- Reference Operator (&) (*Cont.*)

- Consider the following code fragment:

```
andy = 25;  
fred = andy;  
ted = &andy;
```

- The variable that *stores the reference to another variable* (like **ted**) is what we call a **pointer**.
- Pointers are said to "*point to*" the variable whose reference they store.



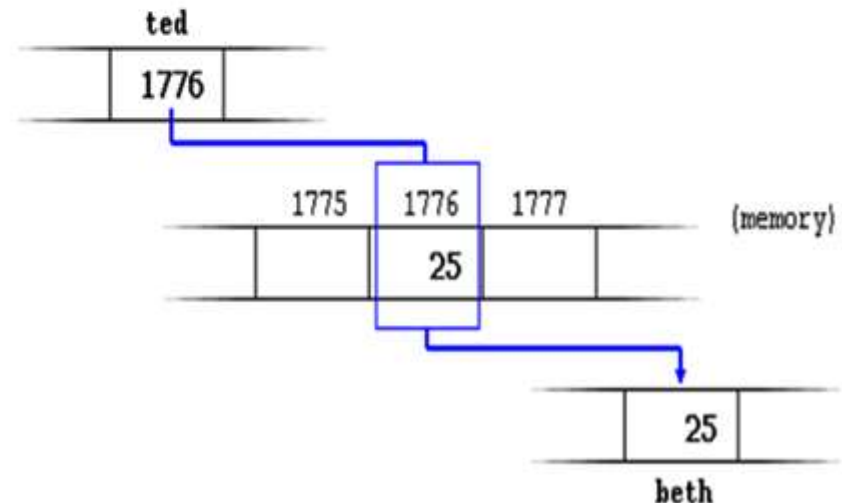
Review C++ Concepts I

6. Pointers(Cont.)- Dereference Operator (*)

- Using a pointer we can directly access the value stored in the variable which it points to.
- To do this, we simply have to precede the pointer's identifier with an asterisk (*), which acts as *dereference operator* and that can be literally translated to "*value pointed by*".

```
beth = *ted;
```

- The expression `ted` refers to the value `1776`, while `*ted` refers to the value stored at address `1776`, which in this case is `25`.



Review C++ Concepts I

6. Pointers(*Cont.*)- Dereference Operator (*) (*Cont.*)

```
andy = 25;  
ted = &andy;
```

```
beth = ted; // beth equal to ted ( 1776 )  
beth = *ted; // beth equal to value pointed by ted ( 25 )
```

- & is the reference operator and can be read as "address of"
- * is the dereference operator and can be read as "value pointed by"

```
andy == 25  
&andy == 1776  
ted == 1776  
*ted == 25
```

Review C++ Concepts I

6. Pointers(*Cont.*)- Declaring Variables of Pointer Types

```
type * name;
```

- where **type** is the data type of the value that the pointer is intended to point to.
- This type is not the type of the pointer itself! but the type of the data the pointer points to.
- ***For example:***

```
int * number;  
char * character;  
float * greatnumber;
```

Review C++ Concepts I

6. Pointers(*Cont.*)- Declaring Variables of Pointer Types (*Cont.*)

- The asterisk sign (*) that we use when *declaring a pointer* only means that it is a *pointer* (**type * name ;**) (*it is part of its type compound specifier*), and should not be confused with the *dereference operator* (**beth = * ted ;**) that we have seen a bit earlier, but which is also written with an asterisk (*).
- They are simply two different things represented with the same sign.

Review C++ Concepts I

6. Pointers(*Cont.*)- Declaring Variables of Pointer Types- *Example-1*

```
// my first pointer
#include <iostream>
using namespace std;
```

```
int main ()
```

```
{
    int firstvalue, secondvalue;
    int * mypointer;
```

pointer

Reference or address

```
mypointer = &firstvalue;
```

```
*mypointer = 10;
```

```
mypointer = &secondvalue;
```

```
*mypointer = 20;
```

```
cout << "firstvalue is " << firstvalue << endl;
```

```
cout << "secondvalue is " << secondvalue << endl;
```

```
return 0;
```

```
}
```

```
firstvalue is 10
secondvalue is 20
```

The value of the variable which its address (or reference) stored in the pointer mypointer. i.e., The value of firstvalue variable.

The value of the variable which its address (or reference) stored in the pointer mypointer. i.e., The value of secondvalue variable.

Review C++ Concepts I

6. Pointers(*Cont.*)- Declaring Variables of Pointer Types- *Example-2*

```
// more pointers
#include <iostream>
using namespace std;

int main ()
{
    int firstvalue = 5, secondvalue = 15;
    int * p1, * p2;

    p1 = &firstvalue; // p1 = address of firstvalue
    p2 = &secondvalue; // p2 = address of secondvalue
    *p1 = 10;          // value pointed by p1 = 10
    *p2 = *p1;         // value pointed by p2 = value pointed by p1

    p1 = p2;           // p1 = p2 (value of pointer is copied)
    *p1 = 20;          // value pointed by p1 = 20

    cout << "firstvalue is " << firstvalue << endl;
    cout << "secondvalue is " << secondvalue << endl;
    return 0;
}
```

firstvalue is 10
secondvalue is 20

firstvalue = 10;
secondvalue = 10;
p1 pointer of secondvalue
secondvalue = 20;

Review C++ Concepts I

6. Pointers(Cont.)- Pointers & Arrays

- The concept of array is very much bound to the *one* of pointer.
- The identifier of an array is equivalent to the address of its first element, *as* a pointer is equivalent to the *address of the first element that it points to*, so in fact they are the same concept.

```
int numbers [20];  
int * p;  
  
p = numbers;
```

Review C++ Concepts I

6. Pointers(Cont.)- Pointers & Arrays- Example

```
// more pointers
#include <iostream>
using namespace std;
```

```
int main ()
```

```
{
    int numbers[5];
    int * p;
    p = numbers; *p = 10;
    p++; *p = 20;
    p = &numbers[2]; *p = 30;
    p = numbers + 3; *p = 40;
    p = numbers; *(p+4) = 50;
    for (int n=0; n<5; n++)
        cout << numbers[n] << ", ";
    return 0;
}
```

10, 20, 30, 40, 50,

Declaration of a pointer **p** of array **numbers**, i.e., it contains the address of the **first** element **numbers[0]** of the array **numbers**.

A pointer **p** of array **numbers**, i.e., it contains the address of the **first** element **numbers[0]**.

Assigning **numbers[0]** by the value **10**

p++ equal **p=number+1**; i.e., mean the pointer of the **second** element, i.e., it becomes the pointer of the element **numbers[1]**. i.e., the pointer **p** contains the address of the element **numbers[1]**

Assigning the value of the element **numbers[1]** to **20**

Pointer of the **third** element **numbers[2]**

Assigning **numbers[2]** by **30**

Assigning **numbers[3]** by **40**

Assigning **numbers[4]** by **50**

p=number+3; means the pointer of the **fourth** element, i.e., it becomes the pointer of the element **numbers[3]**. i.e., the pointer **p** contains the address of the element **numbers[3]**

***(p+4)**; means the pointer of the **fifth** element, i.e., it becomes the pointer of the element **numbers[4]**. i.e., the **p** pointer contains the address of the element **numbers[4]**.

Review C++ Concepts I

6. Pointers(Cont.)- *Pointers Initialization*

```
int number;  
int *tommy = &number;
```

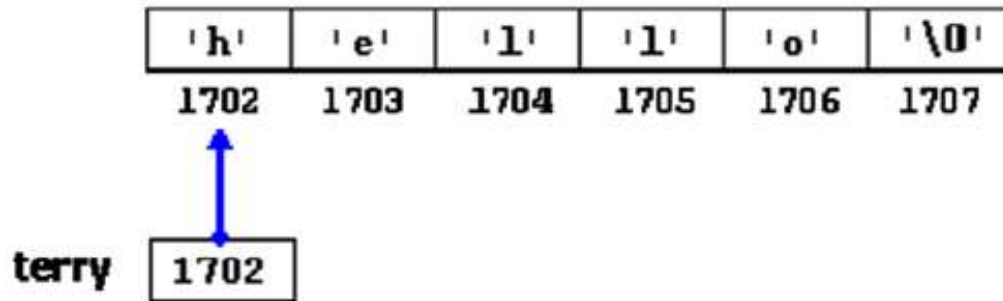
The behavior of this code is equivalent to:

```
int number;  
int *tommy;  
tommy = &number;
```

Review C++ Concepts I

6. Pointers(Cont.)- Pointers Initialization (Cont.)

```
char * terry = "hello";
```



- The pointer `terry` points to a sequence of characters and can be read as if it was an array (*remember that an array is just like a constant pointer*).
- **For example**, we can access the *fifth* element of the array with any of these two expressions:

```
* (terry+4)  
terry[4]
```

Review C++ Concepts I

7. Functions

- Using functions we can structure our programs in a more modular way.
- A **function** is *a group of statements that is executed when it is called from some point of the program.*
- The following is its format:

```
type name ( parameter1, parameter2, ...) { statements }
```

Review C++ Concepts I

7. Functions (Cont.)

- **type** is the data type specifier of the data returned by the function.
- **name** is the identifier by which it will be possible to call the function.
- **parameters** (*as many as needed*): Each parameter consists of a **data type specifier** followed by **an identifier**, like any regular variable declaration (for example: **int x**) and which *acts within the function as a regular local variable*.
 - *They allow to pass arguments to the function when it is called.*
 - The different parameters are separated by commas.
- **statements** is the *function's body*. It is a block of statements surrounded by braces { }.

Review C++ Concepts I

7. Functions (Cont.)- Example

```
// function example
#include <iostream>
using namespace std;

int addition (int a, int b)
{
    int r;
    r=a+b;
    return (r);
}

int main ()
{
    int z;
    z = addition (5,3);
    cout << "The result is " << z;
    return 0;
}
```

The result is 8

Review C++ Concepts I

7. Functions (Cont.)- Scope of Variables

```
#include <iostream.h>
```

```
int Integer;  
char aCharacter;  
char string [20];  
unsigned int NumberOfSons;
```

Global variables

```
main ()  
{
```

```
    unsigned short Age;  
    float ANumber, AnotherOne;
```

Local variables

```
    cout << "Enter your age:"  
    cin >> Age;  
    ...
```

Instructions

```
}
```

Review C++ Concepts I

7. Functions (Cont.)- Scope of Variables (Cont.)- Example

```
// function example
#include <iostream>
using namespace std;

int subtraction (int a, int b)
{
    int r;
    r=a-b;
    return (r);
}

int main ()
{
    int x=5, y=3, z;
    z = subtraction (7,2);
    cout << "The first result is " << z << '\n';
    cout << "The second result is " << subtraction (7,2) << '\n';
    cout << "The third result is " << subtraction (x,y) << '\n';
    z= 4 + subtraction (x,y);
    cout << "The fourth result is " << z << '\n';
    return 0;
}
```

The first result is 5
The second result is 5
The third result is 2
The fourth result is 6

Review C++ Concepts I

7. Functions (*Cont.*)- Function With NO Type- *The use of void*

- Imagine that we want to make a function just to show a message on the screen.
- In this case we should use the **void** type specifier for the function.

```
// void function example
#include <iostream>
using namespace std;

void printmessage ()
{
    cout << "I'm a function!";
}

int main ()
{
    printmessage ();
    return 0;
}
```

I'm a function!

Review C++ Concepts I

7. Functions (Cont.)- Arguments Passed By Value

```
int x=5, y=3, z;  
z = addition ( x , y );
```

```
int addition (int a, int b)
```

```
z = addition ( 5 , 3 );
```



- This way, when the function addition is called, the value of its local variables *a* and *b* become 5 and 3 respectively.
- The arguments passed to the function have been passed *by value*.
- Any modification to either *a* or *b* within the function addition will not have any effect in the values of *x* and *y* outside it, because variables *x* and *y* were not themselves passed to the function, but only copies of their values at the moment the function was called.

Review C++ Concepts I

7. Functions (Cont.)- Arguments Passed By Reference

- There might be some cases where it *needs to manipulate from inside a function the value of an external variable.*
- For that purpose we can use *arguments passed by reference*
- The first thing that should call your attention is that in the declaration of duplicate the type of each parameter was followed by an *ampersand sign (&).*
- This *ampersand* is what specifies that *their corresponding arguments are to be passed by reference instead of by value.*
- *When a variable is passed by reference we are not passing a copy of its value, but we are somehow passing the variable itself to the function and any modification that we do to the local variables will have an effect in their counterpart variables passed as arguments in the call to the function.*

Review C++ Concepts I

7. Functions (*Cont.*)- Arguments Passed By Reference- *Example-1*

```
// passing parameters by reference
#include <iostream>
using namespace std;

void duplicate (int& a, int& b, int& c)
{
    a*=2;
    b*=2;
    c*=2;
}

int main ()
{
    int x=1, y=3, z=7;
    duplicate (x, y, z);
    cout << "x=" << x << ", y=" << y << ", z=" << z;
    return 0;
}
```

x=2, y=6, z=14

When you pass arguments by reference, the function works with references to the original data, meaning any changes made to the parameter will affect the passed argument.

functions that are not supposed to modify the input, to avoid unintended side effects. For such cases, consider passing by constant reference using **const type&**, which ensures that the function cannot modify the argument.

Review C++ Concepts I

7. Functions (*Cont.*)- Arguments Passed By Reference- *Example-2*

- Passing by reference is also an effective way to allow a function to return more than one value.

```
// more than one returning value
#include <iostream>
using namespace std;

void prevnext (int x, int& prev, int& next)
{
    prev = x-1;
    next = x+1;
}

int main ()
{
    int x=100, y, z;
    prevnext (x, y, z);
    cout << "Previous=" << y << ", Next=" << z;
    return 0;
}
```

Previous=99, Next=101

Review C++ Concepts I

7. Functions (Cont.)- Arguments Passed By Pointer- Example-1

- **Passing by Pointer:** Here, the *memory location of the variables* is *passed to the parameters in the function*, and then the operations are performed.

Before Swap

a = 45 b = 35

After Swap with pass by pointer

a = 35 b = 45

```
// C++ program to swap two numbers using
// pass by pointer
#include <iostream>
using namespace std;

void swap(int *x, int *y)
{
    int z = *x;
    *x = *y;
    *y = z;
}

// Driver Code
int main()
{
    int a = 45, b = 35;
    cout << "Before Swap\n";
    cout << "a = " << a << " b = " << b << "\n";

    swap(&a, &b);

    cout << "After Swap with pass by pointer\n";
    cout << "a = " << a << " b = " << b << "\n";
}
```

Review C++ Concepts I

7. Functions (Cont.)- Default Values In Parameters

- When declaring a function it can specify a default value for each of the last parameters.
- This value will be used if the corresponding argument is left blank when calling to the function.

```
// default values in functions
#include <iostream>
using namespace std;

int divide (int a, int b=2)
{
    int r;
    r=a/b;
    return (r);
}

int main ()
{
    cout << divide (12);
    cout << endl;
    cout << divide (20,4);
    return 0;
}
```

6
5

Review C++ Concepts I

7. Functions (Cont.)- Overloaded Functions

- In C++ two different functions can have the same name if their parameter types or number are different.

```
// overloaded function
#include <iostream>
using namespace std;

int operate (int a, int b)
{
    return (a*b);
}

float operate (float a, float b)
{
    return (a/b);
}

int main ()
{
    int x=5,y=2;
    float n=5.0,m=2.0;
    cout << operate (x,y);
    cout << "\n";
    cout << operate (n,m);
    cout << "\n";
    return 0;
}
```

10
2.5

Review C++ Concepts I

7. Functions (Cont.)- Declaring Functions

- All of the functions have defined before the first appearance of calls to them in the source code. These calls were generally in function **main** which we have always left at the end of the source code.
- If you try to repeat some of the examples of functions described so far, but placing the function main before any of the other functions that were called from within it, you will most likely obtain compiling errors.
- To avoid writing the whole code of a function before it can be used in main or in some other function.
- This can be achieved by *declaring just a prototype of the function before it is used, instead of the entire definition.*

Review C++ Concepts I

7. Functions (*Cont.*)- Declaring Functions- *Example*

```
// declaring functions prototypes
#include <iostream>
using namespace std;

void odd (int a);
void even (int a);

int main ()
{
    int i;
    do {
        cout << "Type a number (0 to exit): ";
        cin >> i;
        odd (i);
    } while (i!=0);
    return 0;
}

void odd (int a)
{
    if ((a%2)!=0) cout << "Number is odd.\n";
    else even (a);
}

void even (int a)
{
    if ((a%2)==0) cout << "Number is even.\n";
    else odd (a);
}
```

```
Type a number (0 to exit): 9
Number is odd.
Type a number (0 to exit): 6
Number is even.
Type a number (0 to exit): 1030
Number is even.
Type a number (0 to exit): 0
Number is even.
```